

Trabalho Prático - Chain Blaster

O trabalho deve ser realizado de forma individual e sem o auxílio de ferramentas de IA (Gemini, ChatGPT etc). Se for constatada qualquer evidência de violação, o trabalho receberá nota 0.

1. Descrição do Problema

O objetivo deste trabalho é fazer com que o aluno utilize as técnicas de programação aprendidas na disciplina para desenvolver um jogo eletrônico gráfico semelhante ao *Chain Blaster*. Chain Blaster é um shoot 'em up lançado para o Nintendo 3DS em que o jogador controla uma nave antivírus para impedir um ataque nuclear causado por um supercomputador infectado. O diferencial do jogo está no sistema "Chain Blast", que permite criar reações em cadeia capazes de destruir vários inimigos de uma vez, incentivando pontuações altas e partidas rápidas no estilo arcade. Apesar de ter campanha curta e visual simples, o jogo ficou conhecido entre fãs do gênero pela jogabilidade frenética e foco em combos e ranking online. Veja um exemplo de *gameplay* no vídeo a seguir:

<https://www.youtube.com/watch?v=cpYZN-J3ft8>

Para o TP, você vai implementar uma versão simplificada e modificada do jogo. Em particular:

1. O movimento do jogador é dado pelas teclas A, S, D, W e o campo de força é disparado a partir da tecla de espaço. Isso já está implementado na versão disponibilizada pelo professor (chain.c) no [KIT_DEV_ALLEGRO](#).
2. Todos os inimigos surgem do canto superior da tela e possuem movimento vertical de cima para baixo.
3. Inimigos possuem tamanhos e velocidades aleatórias e podem se sobrepor no espaço, ou seja, não colidem entre eles.
4. Você pode destruir inimigos liberando um campo de força ao seu redor. Se o inimigo encostar no campo de força, ele é destruído.
5. Ao ser destruído, o inimigo também libera um campo de força que é equivalente ao seu tamanho. Inimigos que encostarem nesse campo de força também são destruídos e, por sua vez, liberam também os seus campos de força.
6. Os campos de força têm duração limitada (ex: 2 segundos).
7. Apenas um campo de força pode ser liberado pelo jogador. Para liberar outro, o tempo do primeiro precisa expirar.
8. O tempo joga contra você. A cada *frame*, o jogador perde pontos (ex: 0.1 pontos por *frame*).
9. Para ganhar pontos, o jogador precisa destruir inimigos. Inimigos maiores devem valer mais pontos.
10. O jogo termina quando a nave do jogador é destruída. A tela deve ser congelada por 3 segundos para mostrar a colisão.

11. O jogo registra em um arquivo a maior pontuação histórica (recorde). Se o jogador conseguir ultrapassar o recorde, uma mensagem deve ser impressa na tela com essa informação.
12. É obrigatório exibir a pontuação do jogador durante o jogo. Ao final, uma tela deve mostrar a pontuação do usuário e o recorde atual. Caso a pontuação do usuário seja maior que o recorde, essa informação deve ser exibida e um novo recorde registrado.

Este trabalho vale **15 pontos**. Execute o arquivo `chain_completo.exe` que está no [KIT_DEV_ALLEGRO](#) para ver um exemplo de jogo possível.

2. Critérios de Avaliação

2.1 Solução Apresentada

A avaliação do projeto levará em conta os seguintes critérios:

- Movimentação da nave do jogador deve ser fluida e dentro dos limites da tela (1 ponto);
- Gerenciamento de ao menos 15 inimigos via código (2 pontos);
- Variação de velocidade e dimensões entre os inimigos (1 ponto);
- Eliminação de inimigos por contato com o campo de força do jogador (1 ponto);
- Ativação de um campo de força explosivo ao destruir um inimigo (2 pontos);
- Reação em cadeia: inimigos destruídos por campos de força de outros inimigos (2 pontos);
- Mecânica de redução progressiva da pontuação ao longo do tempo (1 ponto);
- Condição de derrota por colisão direta entre jogador e inimigo (campos de força inimigos não causam dano ao jogador) (2 pontos);
- Interface com exibição da pontuação em tempo real. A pontuação obtida ao derrotar cada inimigo deve ser proporcional ao seu tamanho: quanto maior o inimigo, maior a pontuação recebida (1 ponto);
- Sistema de persistência e visualização do recorde (2 pontos);

2.2 Conhecimento do Código

O conhecimento do aluno será verificado via prova feita em laboratório. Na prova, o aluno deverá fazer modificações no jogo para demonstrar conhecimento do código apresentado. A nota final será **multiplicada pela nota da prova** (nota de 0 a 1).

Exemplo: nota 0.5 na prova oral → nota final dividida por 2.

2.3 Pontos Extras

O professor pode conceder **até 10 pontos extras** por implementações adicionais, como:

- Uso de imagens e animações (sprites);
- Geração de diferentes cenários;
- Tipos variados de tiros e naves com efeitos distintos;
- Sons e músicas;
- Animações para colisões;
- Fases e missões;
- Power-ups (ex: invencibilidade, super tiros);
- Movimentos sofisticados dos inimigos (assim como no jogo original);
- Modo dois jogadores;
- Menus e níveis de dificuldade;
- Qualquer outra funcionalidade criativa.

Importantíssimo: Pontos extras só serão concedidos a quem obtiver **mais de 50% nas provas** (acima de 37,5 pontos no total das três provas).

3. Como eu faço?

Apesar da descrição parecer complexa, o trabalho é simples. Com os conhecimentos adquiridos na disciplina e uma introdução ao desenvolvimento gráfico, o aluno é capaz de implementar o jogo.

Para isso, será necessário usar a biblioteca **Allegro5**, que fornece funções gráficas úteis. Se você for implementar o jogo no Windows, basta seguir os seguintes passos:

1. Extraia o conteúdo do [KIT_DEV_ALLEGRO](#).
2. Copie o arquivo allegro-5.0.10-mingw-4.7.0.zip para a pasta raiz (C:\) e extraia o seu conteúdo (“extract here” ou “extraia aqui”). Depois de extrair, você deve ter uma pasta “C:\allegro-5.0.10-mingw-4.7.0\” e dentro dela as pastas “bin”, “lib” e “include”.
3. Copie e cole o restante dos arquivos do [KIT_DEV_ALLEGRO](#) para a pasta que vai usar para desenvolver o seu jogo.
4. Execute o comando “make” no prompt de comando para compilar e gerar o executável do jogo.

Tutoriais para instalar a Allegro5 com MinGW no Windows:

- <https://www.youtube.com/watch?v=AezxBP687n8>
- <https://www.youtube.com/watch?v=cggjzJzm00w>

4. Roteiro de Desenvolvimento Sugerido

Faça o download do [KIT_DEV_ALLEGRO](#) disponível em:

https://www.dropbox.com/scl/fi/q5znr3ypnwfgvktbtwbgI/KID_DEV_ALLEGRO.zip?rlkey=rn3gwgngq3clrd4uh26yfs7vu&dl=0

Etapas sugeridas, em ordem:

1. Compilar e rodar a versão básica do jogo (chain.c) que está no [KIT_DEV_ALLEGRO](#).
2. Restringir o movimento da nave do jogador, fazendo que ela não ultrapasse os limites da tela.
3. Criar as estruturas dos inimigos. Inicializar elas e implementar a atualização das suas coordenadas à medida que o tempo passa.
4. Desenhar as estruturas na tela.
5. Implementar colisão dos inimigos com a nave do jogador. Garantir que o jogo termine quando o jogador colidir com qualquer inimigo.
6. Implementar colisão dos inimigos com o campo de força do jogador.
7. Implementar o disparo do campo de força dos inimigos quando estes são destruídos.
8. Implementar a colisão dos inimigos com os seus campos de força, o que permite a reação em cadeia de destruição (chain).
9. Implementar e exibir a pontuação. Considere que a pontuação aumenta quando inimigos são destruídos e decresce levemente com o tempo.
10. Implementar armazenamento do recorde;
11. Adicionar funcionalidades extras;

5. Aulas para começar o desenvolvimento do TP

Assista à aula a seguir para entender como a biblioteca Allegro funciona:

[Introdução à programação em Allegro](#)

Assista à aula a seguir para iniciar o desenvolvimento do seu TP:

[Início da programação de um protótipo do jogo Space Invaders em Allegro](#)

Mais vídeos sobre Allegro feitos pelo professor Pedro Olmo (veja a aula do jogo R-Type!):

[\(6\) Vídeos sobre Allegro - YouTube](#)